Person 2 — ETA & Prediction Engine

Team brief

What this part does, in one line

The brain of the server. It takes the raw GPS pings the bus sends and figures out *where the bus actually is*, *when it'll reach each upcoming stop*, and *whether anything is wrong*.

Where it sits in the system

Bus -> Person 1 (data ingestion + DB) -> **Person 2 (the ETA engine)** -> DB -> Person 3 (SMS / IVR / dashboard / display board).

The engine doesn't talk to the passenger directly; it produces the numbers that Person 3's layer reads aloud or displays.

The four jobs

1. ETA calculation. "How many minutes until Bus 17 reaches the next stop?" Done with a simple distance ÷ speed on day one, then a smarter version that learns each segment of the route over time.

2. Network-loss prediction. Rural areas drop signal constantly. When the bus stops sending updates, the engine *guesses* where it is based on its last known speed and how long it's been silent. Status flips from LIVE -> ESTIMATED -> LAST_KNOWN as the blackout grows.

3. Historical learning. Every completed trip teaches the system how long each road segment actually takes — broken down by time of day (so 8 AM rush hour and midnight are learned separately). Over weeks, the ETA gets noticeably more accurate without any manual tuning.

4. Route adherence (geofencing). The engine draws an invisible 50-metre corridor around each route. If the bus drifts outside it for three packets in a row, that's a divert — and the alert pipeline fires: log it in the DB, flash a banner on the authority dashboard, and **automatically SMS / IVR-call every passenger who recently asked about that bus** in their own language. That last bit is what plugs into Person 3's IVR/SMS work directly.

What gets written to the database (Person 3 reads these)

- Bus position
- The next stop
- ETAs for every upcoming stop
- A status flag (LIVE / ESTIMATED / LAST_KNOWN / OFF_ROUTE)

- A confidence number from 0 to 1 so the display can say “ETA 12 min” vs “ETA ~12 min (estimated)”

What the engine needs from the rest of the team

- **Person 1** -> the canonical route data (list of stops + the polyline of the actual road) and the live stream of cleaned GPS packets.
- **Person 3** -> a small `queries` log of which passengers asked about which bus, so the divert pipeline knows who to message.

The cold-start problem — what about day one when there’s no history?

Fair question: the historical learning only kicks in once trips are on the books. Day one, the table is empty. Four ways to handle it, used in combination:

1. **Hardcoded default speed.** ~30 km/h for rural highways, ~25 for mixed, ~15 for town centres. Works on switch-on, not accurate, but the display has something to show from minute one.
2. **Seed from the published bus schedule.** Most routes already have a paper schedule (“leaves at 14:00, reaches Tollgate 14:08”). That’s exactly the data needed — read it once, pre-fill the segment averages.
3. **Seed from OpenStreetMap / OSRM.** OSRM is a free routing service that, given two GPS points, tells you how long a car would take to drive between them on the real road network. One lookup per segment at route-load time, answer goes into the table.
4. **One test drive per route.** Stick the device on a bus, run the route once, record the timestamps. Most accurate seed possible. Logistics-heavy but unbeatable.

The engine **stacks all four** via a fallback chain: if this bucket’s average has enough samples use it; otherwise fall back to the segment average; otherwise the route average; otherwise the hardcoded default. The system never fails to produce an ETA — it just gets more accurate as the table fills in.

Bonus trick: for the first ~10 observations in a new bucket, the smoothing weight α gets bumped from 0.25 to 0.5 so the system adapts fast while data is thin, then settles down. And for the first ~2 weeks per route, the display can tag ETAs as “estimate (still learning)” so passengers don’t trust them as more precise than they are.

Decision to make as a team: which of the four seeds to use? Suggested: schedule data first, OSRM lookup as the automated fallback, test drives only when both are unavailable.

Where does the route data (polyline) come from?

The engine needs the canonical route — a list of stops plus the actual road geometry connecting them. Five possible sources, in practical order:

1. **OpenStreetMap + OSRM (recommended default).** OSRM is a free routing service on top of OpenStreetMap. One HTTP call per stop-pair returns the driving polyline between them. Concatenate for the full route. Automated, free, works anywhere OSM covers. Bonus: also gives a travel-time estimate that seeds the cold-start table.

2. **Manual digitisation.** For routes where OSM is missing the road, someone traces it by hand in Google My Maps or QGIS and exports a GeoJSON LineString. ~30 minutes per route. Fine for a pilot of 5–10 routes.
3. **Test drive.** Stick the device on a bus, drive the route once, the recorded GPS trail *is* the polyline — by definition the most accurate possible. Also seeds real travel times for free.
4. **GTFS feeds.** If the transport authority publishes routes as GTFS data, `shapes.txt` is the polyline file. Authoritative. Most rural Indian operators don't publish GTFS yet, but worth checking.
5. **Google Maps Directions API.** Better coverage than OSM in some places, but paid (~$5 per 1k requests) and has redistribution restrictions.

Recommended pipeline for the pilot: OSRM auto-fetch first -> manual digitisation for routes where OSRM fails -> optional test drive per route to validate and seed historical data.

This sits squarely in Person 1's territory — they own the database and the route-loading scripts. The engine just reads the polyline rows; it doesn't care where they came from.

Deliverables for the presentation

Three flow diagrams (ETA workflow, prediction workflow, divert-alert pipeline) plus the geofence equations. All four are already drafted.

Bottom line

Person 1 collects the raw data, Person 3 talks to the passenger, and Person 2 is the layer in between that turns "lat/lon every 2–3 minutes" into "Bus 17 arriving in 8 minutes, and by the way it just went off route". Without this layer there's nothing meaningful to display, speak, or text.